Traffic Light Final Project

Embedded Operating Systems

CDA3631.01I&T Spring 2026

Partners: Avery Martinez and Carlos Medina

Table of Contents

a. Introduction (pg. 3)
b. Objectives (pg. 3)
c. Bill of Materials (pg. 3)
d. Methods (pg. 3 - ____)
e. Conclusion (pg. ___)
f. GitHub Repo (pg. _)
g. References (pg. ___)

a. Introduction

Embedded Operating Systems is a specialized software platform designed to manage the hardware and software resources of a dedicated computing system, often operating with very strict limitations such as the memory, lack of processing power, and the ability to use its energy to power high end projects or devices. These systems are the brain behind everything from the automotive engine block controllers and household appliances to the industrial robots that are being produced around the world. They also function as Real-Time Operating Systems (RTOS) that ensure that critical tasks are processed within the precise time measures of milliseconds that means the difference between a successful operation or a complete system failure. This project exemplifies the precision needed by the RTOS to ensure the timing of a set of tasks can properly run a traffic light intersection.

b. Objectives

- **Utilize tasks and correctly implement a queue for said tasks.**
- **Understand the importance of precise timing for traffic safety purposes.**
- **Successfully implement a "walk" button to interrupt with higher priority than the standard loop.**

c. Bill of Materials

Push Button (One on Nucleo F446-re)	2x	~$0.10
Red LED	4x	~$0.40
Yellow LED	4x	~$0.40
Green LED	4x	~$0.40
Blue LED	2x	~$0.20
Resistors (330 Ω)	14x	$0.30
STM32 Nucleo F446-RE	1x	$16.00
TOTAL	31x	$17.80

d. Methods

The purpose of this project was to display the functionality of a simple traffic light intersection. Although it seems simple, the group had to utilize a few different methods to ensure that the LEDs would run precisely and effectively. Since the majority of Embedded OS are Real-Time Operating Systems, the use of tasks and queues goes without question.

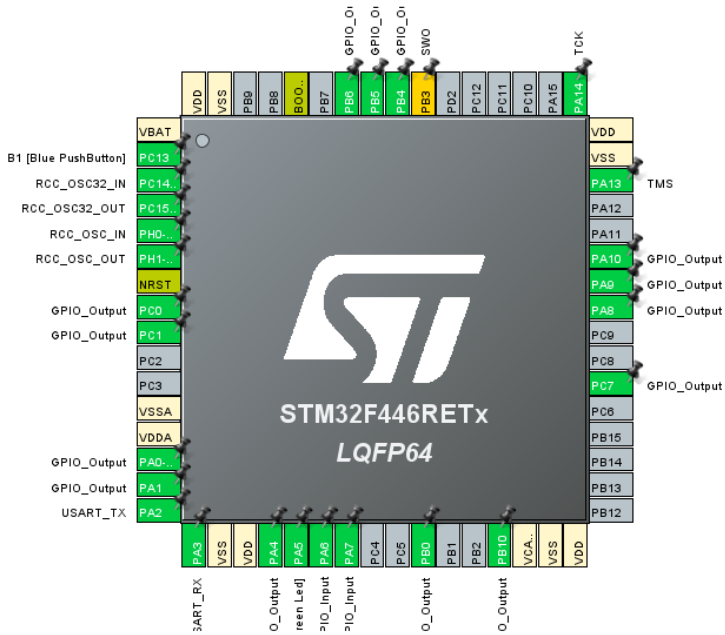


Figure 1: Pinout Configuration

The following image above displays the proper Nucleo F446RE pinout configuration. As observed, there were 14 total pins that were set to GPIO_Output given the 14 total LEDs. In addition, there were two pins that were set to GPIO_Input for the buttons to receive input. This allows for the microcontroller to receive a signal that will act as the pedestrian signal placed on the corners of each intersection.

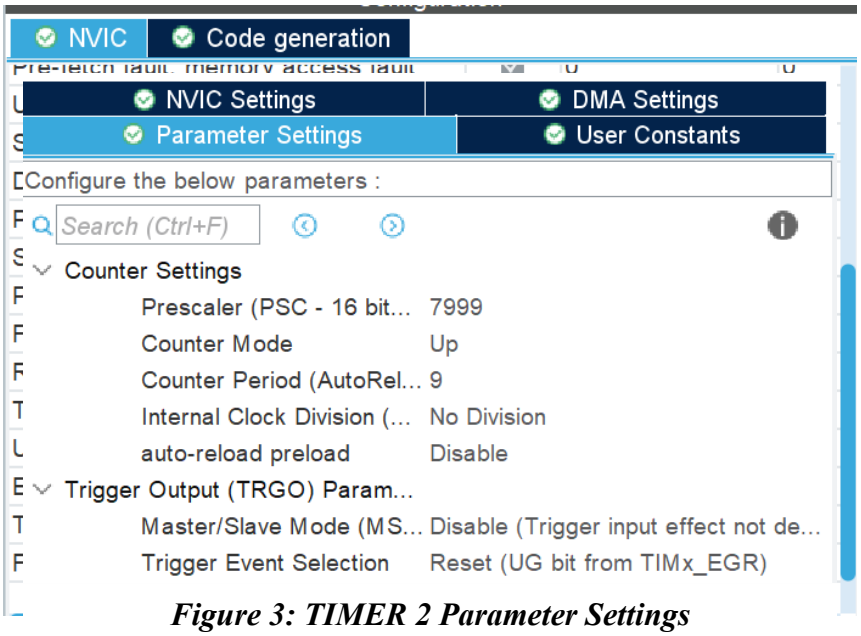


Figure 3: TIMER 2 Parameter Settings

The images shown above are the pinout &

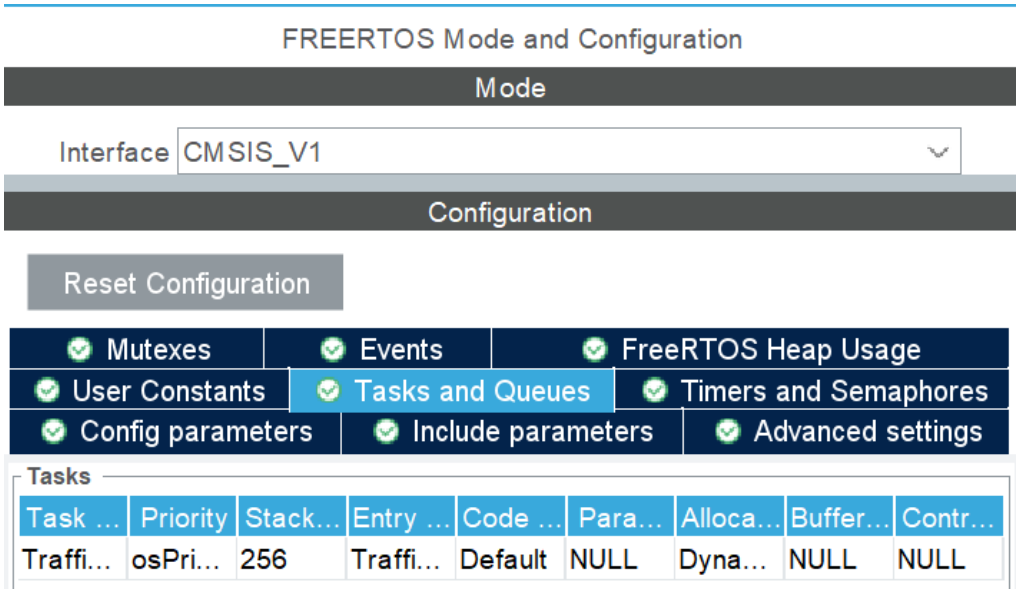
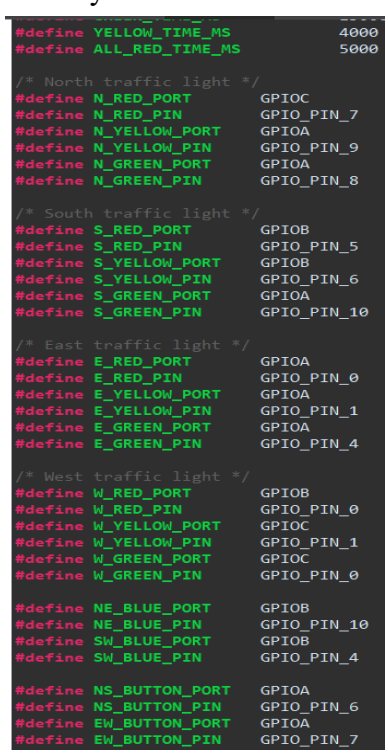


Figure 4: FREERTOS Task Configuration

configurations of each setting. In figure 2, the group agreed upon using Timer 2 global interrupt instead of having to individually ensure the proper timing of each set of LEDs. In the following image, *Figure 3,* the timer parameters were given as a prescaler of 7999 with the counter mode set to up and a Counter Period of 9. As stated before, this ensures that each set of the LEDs, including the pedestrian light, is precisely timed for an accurately running traffic intersection.



```
#define YELLOW_TIME_MS          4000
#define ALL_RED_TIME_MS         5000

/* North traffic light */
#define N_RED_PORT        GPIOC
#define N_RED_PIN         GPIO_PIN_7
#define N_YELLOW_PORT     GPIOA
#define N_YELLOW_PIN      GPIO_PIN_9
#define N_GREEN_PORT      GPIOA
#define N_GREEN_PIN       GPIO_PIN_8

/* South traffic light */
#define S_RED_PORT        GPIOB
#define S_RED_PIN         GPIO_PIN_5
#define S_YELLOW_PORT     GPIOB
#define S_YELLOW_PIN      GPIO_PIN_6
#define S_GREEN_PORT      GPIOA
#define S_GREEN_PIN       GPIO_PIN_10

/* East traffic light */
#define E_RED_PORT        GPIOA
#define E_RED_PIN         GPIO_PIN_0
#define E_YELLOW_PORT     GPIOA
#define E_YELLOW_PIN      GPIO_PIN_1
#define E_GREEN_PORT      GPIOA
#define E_GREEN_PIN       GPIO_PIN_4

/* West traffic light */
#define W_RED_PORT        GPIOB
#define W_RED_PIN         GPIO_PIN_0
#define W_YELLOW_PORT     GPIOC
#define W_YELLOW_PIN      GPIO_PIN_1
#define W_GREEN_PORT      GPIOC
#define W_GREEN_PIN       GPIO_PIN_0

#define NE_BLUE_PORT      GPIOB
#define NE_BLUE_PIN       GPIO_PIN_10
#define SW_BLUE_PORT      GPIOB
#define SW_BLUE_PIN       GPIO_PIN_4

#define NS_BUTTON_PORT    GPIOA
#define NS_BUTTON_PIN     GPIO_PIN_6
#define EW_BUTTON_PORT    GPIOA
#define EW_BUTTON_PIN     GPIO_PIN_7
```

Figure 5: Pin & Port definitions

```c
typedef enum
{
    STATE_NS_GREEN = 0,
    STATE_NS_YELLOW,
    STATE_ALL_RED_TO_EW,
    STATE_EW_GREEN,
    STATE_EW_YELLOW,
    STATE_ALL_RED_TO_NS
} TrafficState;

typedef enum
{
    EVENT_TIMER_TICK = 0
} TrafficEvent;

QueueHandle_t trafficQueue;
SemaphoreHandle_t pedMutex;
TrafficState currentState = STATE_NS_GREEN;
volatile uint32_t stateElapsedMs = 0;
/* USER CODE END PM */

/* Private variables ----------------------
TIM_HandleTypeDef htim2;

UART_HandleTypeDef huart2;

osThreadId TrafficTaskHandle;
/* USER CODE BEGIN PV */
volatile uint8_t ns_ped_request = 0;
volatile uint8_t ew_ped_request = 0;
/* USER CODE END PV */
```

Figure 6: Queue & Mutex Definition

```c
void AllLightsRed(void)
{
    HAL_GPIO_WritePin(N_RED_PORT, N_RED_PIN, GPIO_PIN_SET);
    HAL_GPIO_WritePin(S_RED_PORT, S_RED_PIN, GPIO_PIN_SET);
    HAL_GPIO_WritePin(E_RED_PORT, E_RED_PIN, GPIO_PIN_SET);
    HAL_GPIO_WritePin(W_RED_PORT, W_RED_PIN, GPIO_PIN_SET);

    HAL_GPIO_WritePin(N_YELLOW_PORT, N_YELLOW_PIN, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(S_YELLOW_PORT, S_YELLOW_PIN, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(E_YELLOW_PORT, E_YELLOW_PIN, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(W_YELLOW_PORT, W_YELLOW_PIN, GPIO_PIN_RESET);

    HAL_GPIO_WritePin(N_GREEN_PORT, N_GREEN_PIN, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(S_GREEN_PORT, S_GREEN_PIN, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(E_GREEN_PORT, E_GREEN_PIN, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(W_GREEN_PORT, W_GREEN_PIN, GPIO_PIN_RESET);
}

void SetNSGreen(void)
{
    AllLightsRed();

    HAL_GPIO_WritePin(N_RED_PORT, N_RED_PIN, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(S_RED_PORT, S_RED_PIN, GPIO_PIN_RESET);

    HAL_GPIO_WritePin(N_GREEN_PORT, N_GREEN_PIN, GPIO_PIN_SET);
    HAL_GPIO_WritePin(S_GREEN_PORT, S_GREEN_PIN, GPIO_PIN_SET);
}

void SetNSYellow(void)
{
    AllLightsRed();

    HAL_GPIO_WritePin(N_RED_PORT, N_RED_PIN, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(S_RED_PORT, S_RED_PIN, GPIO_PIN_RESET);
```

Figure 7: Traffic Light Functions

```c
osThreadDef(TrafficTask, Traffic_Task, osPriorityNormal, 0, 256);
TrafficTaskHandle = osThreadCreate(osThread(TrafficTask), NULL);

// Pedestrian Button Task
osThreadDef(PedTask, PedestrianTask, osPriorityBelowNormal, 0, 128);
osThreadCreate(osThread(PedTask), NULL);

// LED Blink Task
osThreadDef(BlinkT, BlinkTask, osPriorityLow, 0, 128);
osThreadCreate(osThread(BlinkT), NULL);
/* USER CODE END RTOS_THREADS */
```

Figure 8: Thread Creation with Priority

```c
HAL_Init();

SystemClock_Config();

MX_GPIO_Init();
MX_USART2_UART_Init();
MX_TIM2_Init();

/* USER CODE BEGIN 2 */

/* USER CODE END 2 */

/* USER CODE BEGIN RTOS_MUTEX */
pedMutex = xSemaphoreCreateMutex();
/* USER CODE END RTOS_MUTEX */

/* USER CODE BEGIN RTOS_SEMAPHORES */
/* add semaphores, ... */
/* USER CODE END RTOS_SEMAPHORES */

/* USER CODE BEGIN RTOS_TIMERS */
/* start timers, add new ones, ... */
/* USER CODE END RTOS_TIMERS */

/* USER CODE BEGIN RTOS_QUEUES */
/* add queues, ... */
trafficQueue = xQueueCreate(10, sizeof(TrafficEvent));
```

Figure 9: Mutex & Queue in RTOS

```c
void BlinkTask(void const * argument)
{
  for(;;)
  {
    /* North-South Blue LED Logic */
    // Only blink if there is a request AND NS is currently Green or Yellow
    if (ns_ped_request == 1 && (currentState == STATE_NS_GREEN || currentState == STATE_NS_YELLOW))
    {
      HAL_GPIO_TogglePin(NE_BLUE_PORT, NE_BLUE_PIN);
    }
    else
    {
      HAL_GPIO_WritePin(NE_BLUE_PORT, NE_BLUE_PIN, GPIO_PIN_RESET);
    }

    /* East-West Blue LED Logic */
    // Only blink if there is a request AND EW is currently Green or Yellow
    if (ew_ped_request == 1 && (currentState == STATE_EW_GREEN || currentState == STATE_EW_YELLOW))
    {
      HAL_GPIO_TogglePin(SW_BLUE_PORT, SW_BLUE_PIN);
    }
    else
    {
      HAL_GPIO_WritePin(SW_BLUE_PORT, SW_BLUE_PIN, GPIO_PIN_RESET);
    }

    osDelay(500); // 500ms toggle = 1 second cycle
  }
}
```

Figure 10: RTOS Blink Task

The collection of images displayed above are the snip bits of code that are put together and allow the entire traffic light system to function properly. In *figure 5*, the list of pins and ports that were used to configure the GPIO were defined based on the pins used for the LEDs. In the following figure, we created the queue and mutex definitions so that the entire program could utilize both. The mutex would be used for each of the tasks, one task for the traffic lights and one for the button task. Whenever a task sends a request, it will grab possession of the mutex and if another task requests it will enter the queue and wait until the given time frame to obtain possession of said mutex. This allows for proper and safe procedures within the Real-Time Operating System. In *figure 8*, the creation of the tasks provided by the configurations set in FREERTOS allows for the functions provided before the main function to work effectively. Each task also contains the current state of the microcontroller that will update in the RTOS. This keeps track of which task is currently running in the background. *Figure 10* is simply another snip bit of code containing the blinking task. This task determines the precise timing of the blinking functionality. The blue LED will blink every second only if the side that it corresponds to is currently not at least red. On the other hand, if the button is pressed on the perpendicular side, the traffic lights will turn yellow within two seconds and finish its blinking task while also releasing the mutex to the other task. And this process continues forever.

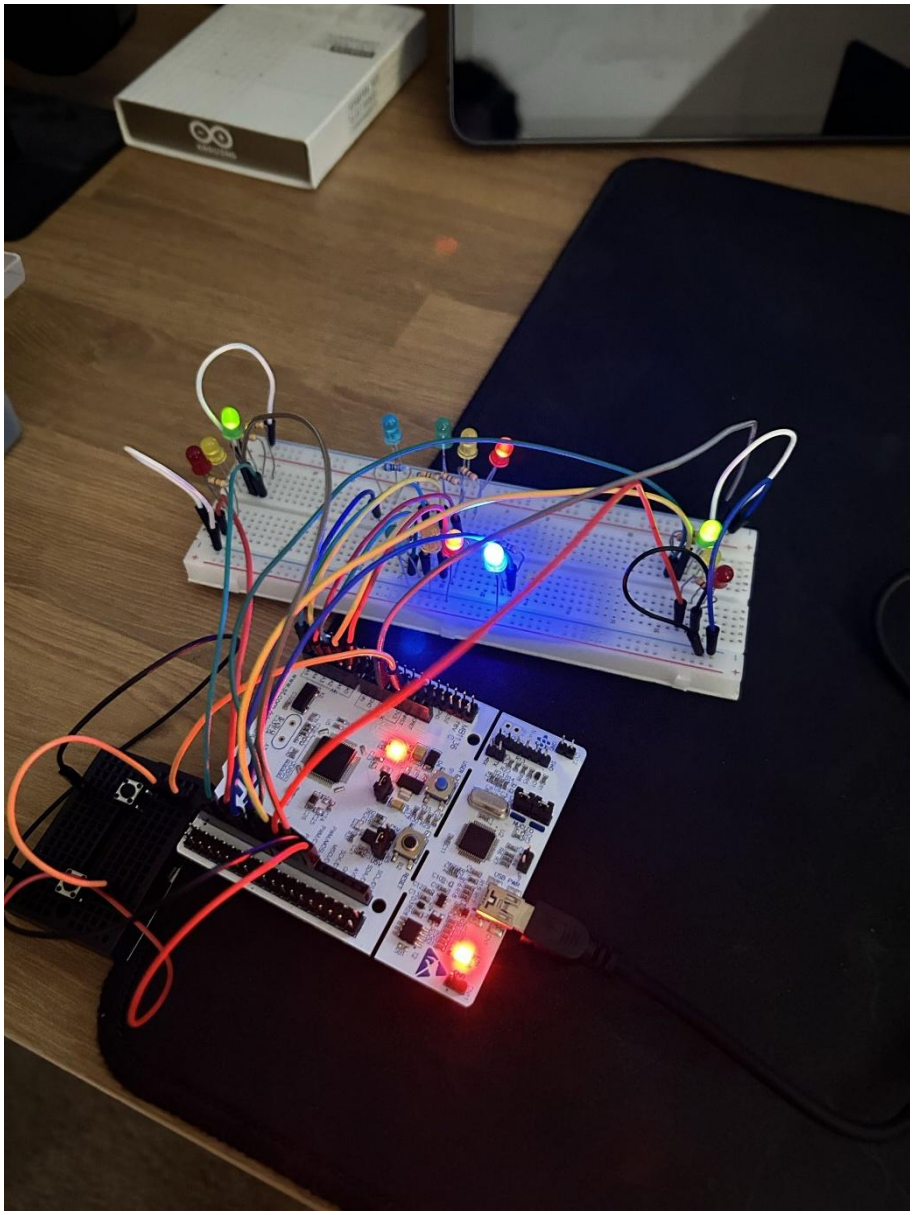


Figure 11: Finished Circuit Design

e. Results and Discussion

The traffic light system was successfully implemented using the STM32 microcontroller and FreeRTOS, and it operated correctly in real time. The system cycled through all traffic light states as expected, including green, yellow, and all-red phases for both North-South and East-West directions. Pedestrian button inputs were properly detected, and the system adjusted the timing when a request was made. The use of TIM2 provided accurate timing, while the queue ensured

efficient communication between the interrupt and the TrafficTask. Additionally, the mutex helped protect shared variables and prevented race conditions between tasks. The use of FreeRTOS improved system organization by separating functionality into different tasks. One limitation of the system is the use of fixed timing values, which could be improved in the future by adding sensors for adaptive traffic control. Overall, the project demonstrates a functional and well-structured real-time embedded system.

f. Conclusion

In conclusion, this code represents a sophisticated and safety-critical embedded system designed using modular, event-driven architecture. By utilizing a Real-Time Operating System (FREERTOS), the program successfully balances high-priority tasks and safety timing with low-priority interactions. Instead of using single loop, such as the *while* loop contained inside the main function, the code is broken down into multiple parts to break down the intersection's responsibilities into specialized, user-defined tasks. This allowed for a smooth and reactive system that ensures the RTOS influences never compromise the safety rules put in place by many cities and countries.

g. GitHub Repo

https://github.com/avery0201/Embedded-OS-Final-Project

h. Acknowledgement

I would like to express my sincere gratitude to my professor, Dr. Hoan T. Ngo, for his guidance, support, and valuable feedback throughout this project. His instruction in embedded systems and real-time programming was essential for the successful completion of this work. I also appreciate the resources and tools provided, which helped me develop practical skills in STM32 and FreeRTOS. Finally, I would like to thank my classmates for their support and discussions, which contributed to a better understanding of the project.

i. References

[1] B. Lutkevich, B. Posey, and I. Wigmore, "What is an embedded operating system?" Search IoT, https://www.techtarget.com/iotagenda/definition/embedded-operating-system (accessed Apr. 15, 2026).

[2] Technical documentation, https://docs.nordicsemi.com/bundle/ncs-1.2.1/page/zephyr/boards/arm/nucleo_f446re/doc/index.html (accessed Apr. 19, 2026).